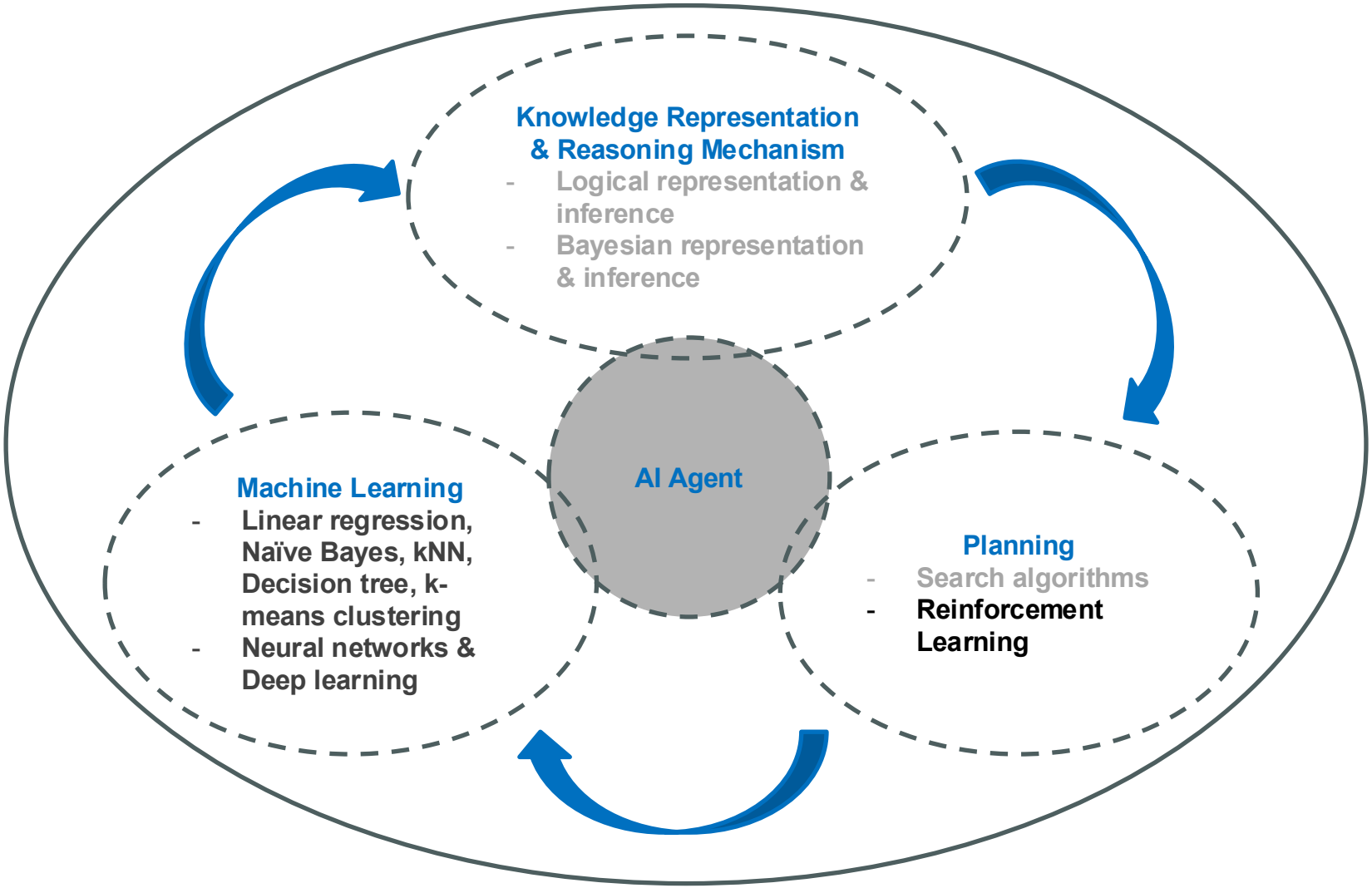
MONASH University
Information Technology



FIT3080 – Artificial Intelligence

Machine Learning – Part 2
Artificial Neural Network
Chapters 19.6.1-19.6.4, 22.1-22.3

Overview of Unit topics



Learning Objectives

- **Machine learning – definition**
- **Approaches to learning from data**
- **Mathematical principles**
- **Supervised machine learning**
  - Decision Trees
  - Regression
  - Linear classifiers with thresholds
  - Logistic regression
  - Artificial Neural Networks
  - Naïve Bayes
  - k Nearest Neighbour (k-NN)
- **Unsupervised learning**
  - Clustering
    - > Similarity measures
    - > The K-means algorithm

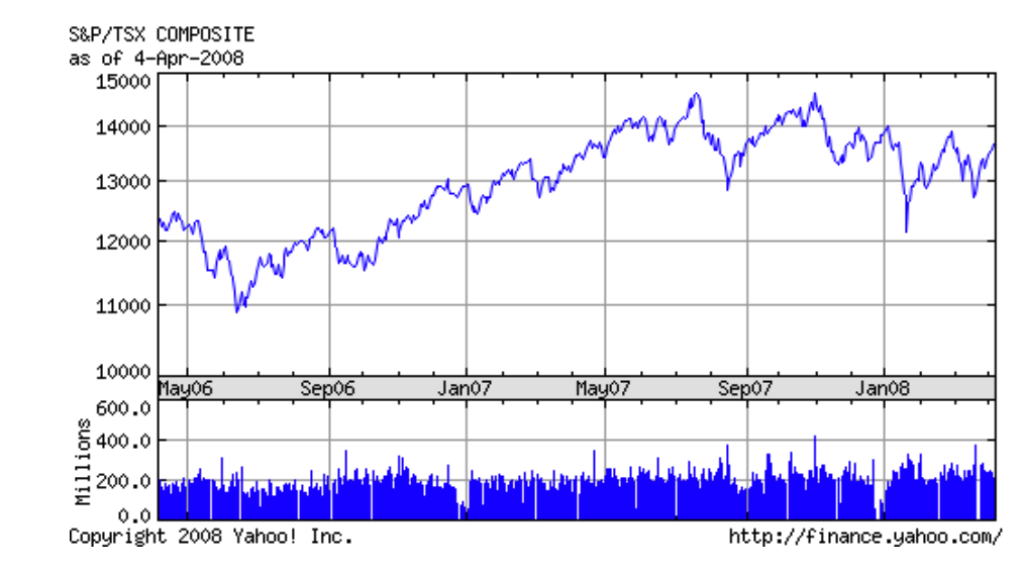
MONASH University
Information Technology



FIT3080 – Artificial Intelligence

Regression

Regression



- **Given a training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)\}$**
  - $\mathbf{x}_j$ and y_j are continuous → regression
  - assume $\mathbf{x}_j \in R^m, y_j \in R$
- **E.g., y_j is stock price, $\mathbf{x}_j$ contains company profit, debt, cash flow, …**

Univariate Linear Regression

- **Given a training set $\{(x_1, y_1), \ldots, (x_N, y_N)\}$**
  - assume $x_j \in R, y_j \in R$

$$h_{\mathbf{w}}(x) = w_1 x + w_0$$

  - we want to learn the parameters w_0 and w_1

- **Loss: Square of the difference between the true and predicted target value for x_j**

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^{N} (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^{N} (\underbrace{y_j}_{\text{truth}} - (\underbrace{w_1 x_j + w_0}_{\text{prediction}}))^2$$

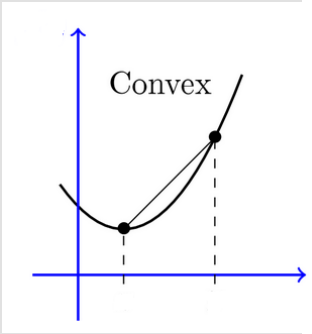
where *N* is the number of samples

Learning the Parameters

- **Find $\mathbf{w}^*$ that minimizes the loss function**

$$\mathbf{w}^* = \text{argmin}_{\mathbf{w}} Loss(h_{\mathbf{w}})$$

  - $Loss(h_{\mathbf{w}})$ is a convex function, so $\mathbf{w}^*$ is unique



- **How to find $\mathbf{w}^*$?**
  - Set the derivatives to zero: $\frac{\partial}{\partial w_i} Loss(h_{\mathbf{w}}) = 0$
    - > For a linear function $h_{\mathbf{w}} = w_1 x + w_0$ the solution is:

$$w_0 = \frac{\sum y_j - w_1 \sum x_j}{N} \qquad w_1 = \frac{N \sum x_j y_j - \sum x_j \sum y_j}{N \sum x_j^2 - (\sum x_j)^2}$$

  - Use an iterative algorithm such as ***gradient descent***

Univariate Linear Regression – Example (I)

$x_1, y_1 \quad x_2, y_2 \quad x_3, y_3$

- **Training data:** $\{(1,3),(2.1,0.5),(-5,6.2)\}$
- **Linear regression model:** $h_{\mathbf{w}} = w_1 x + w_0$
- **Loss function:**

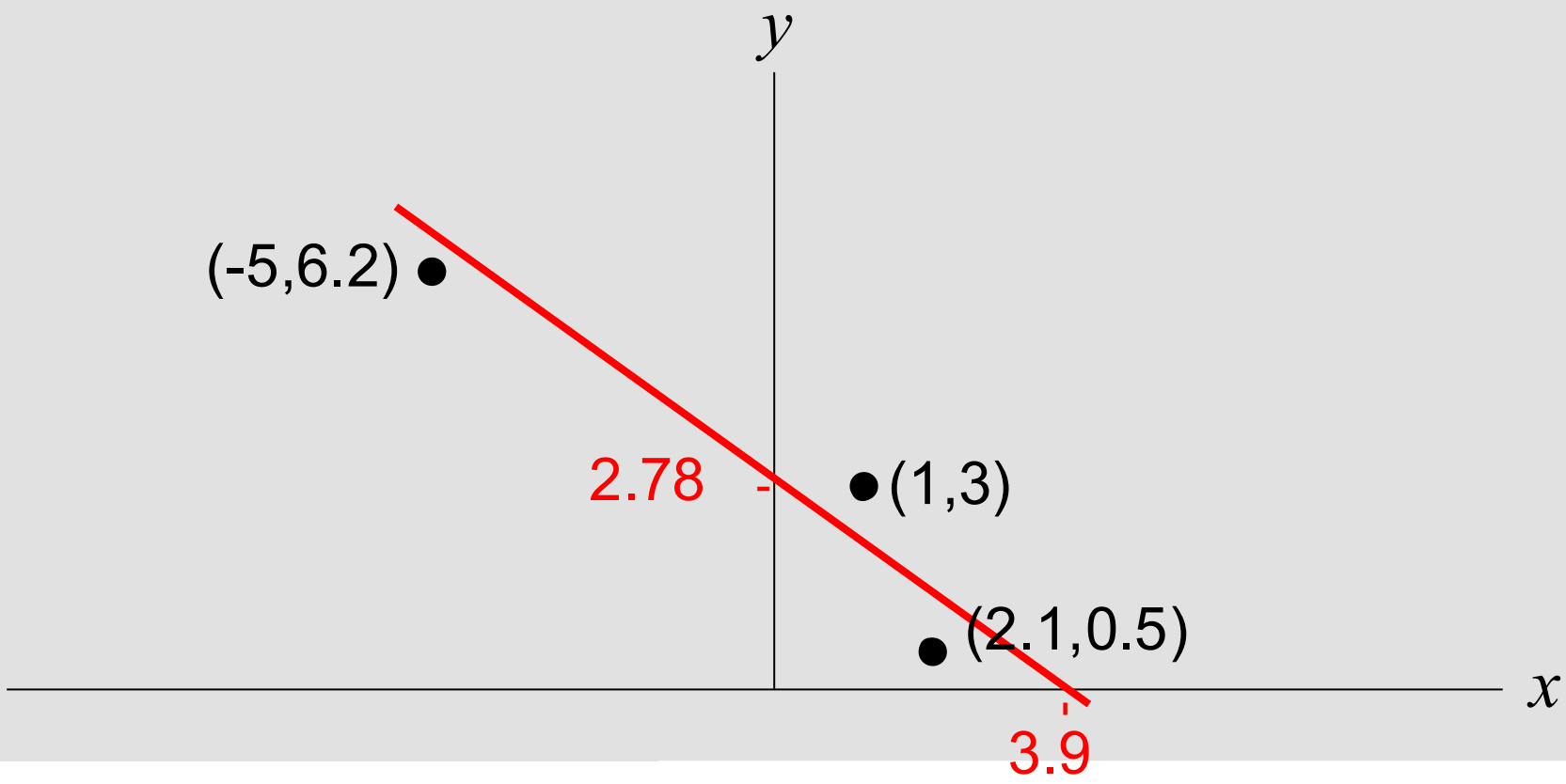
$$Loss(h_{\mathbf{w}}) = (3 - (1w_1 + w_0))^2 + (0.5 - (2.1w_1 + w_0))^2 + (6.2 - (-5w_1 + w_0))^2$$

$$w_0 = \frac{(3+0.5+6.2) - w_1(1+2.1-5)}{3} = \frac{9.7 + w_1 1.9}{3} = 2.78$$

$$w_1 = \frac{3(1\times 3 + 2.1\times 0.5 + (-5)\times 6.2) - (1+2.1-5)(3+0.5+6.2)}{3(1^2 + 2.1^2 + (-5)^2) - (1+2.1-5)^2} = -0.712$$

Univariate Linear Regression – Example (II)

- **Training data:** {(1,3),(2.1,0.5),(−5,6.2)}
- **Function:** $h_w = -0.712x + 2.78$



Gradient Descent (GD) Algorithm

1. **$\mathbf{w}^0 \leftarrow$ any point in the parameter space; t = 1**
2. **Until convergence ($|| \mathbf{w}^t - \mathbf{w}^{t-1} || < \varepsilon$) do**
   a. For each w_i in **w** do

Learning rate

$$w_i^t \leftarrow w_i^{t-1} - \alpha \frac{\partial}{\partial w_i} Loss(h_{\mathbf{w}})$$

Index in vector **w**

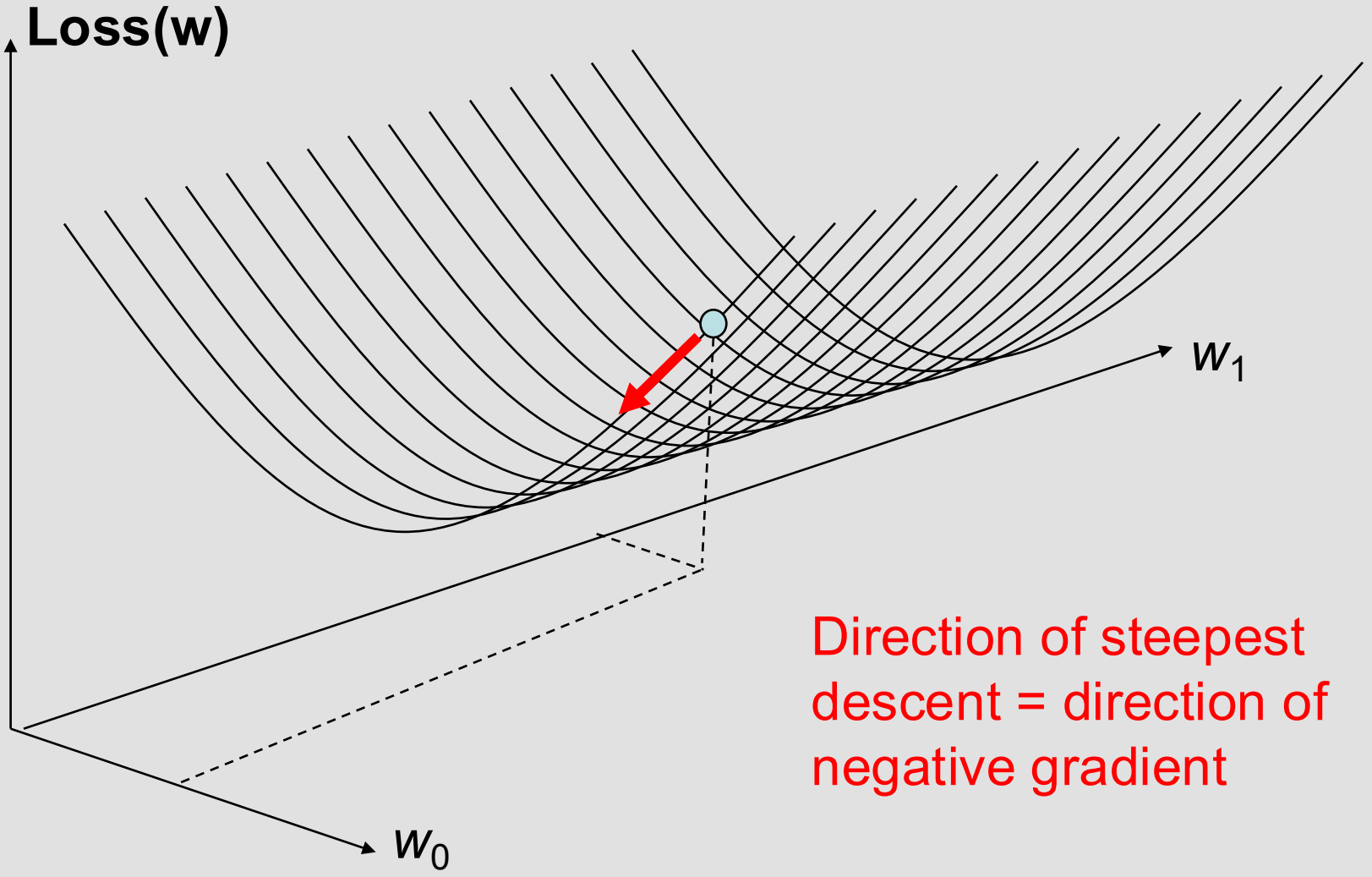
$$b.\ t \leftarrow t + 1$$

where for univariate linear regression

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^{N} \left(y_j - h_{\mathbf{w}}(x_j)\right)^2$$

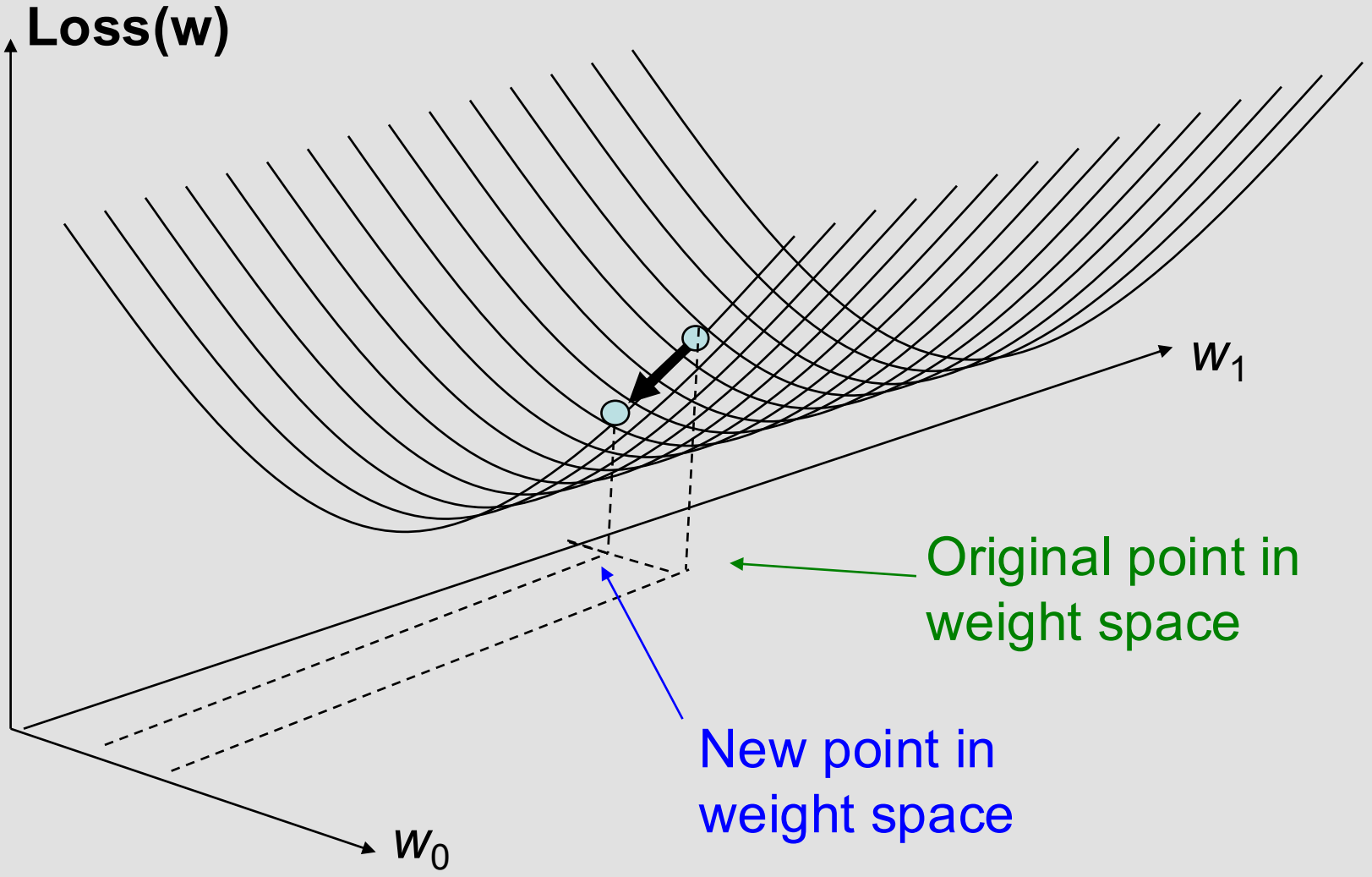
$$\frac{\partial}{\partial w_i} Loss(h_{\mathbf{w}}) = -2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} h_{\mathbf{w}}(x)$$

Illustration of Gradient Descent (I)



Direction of steepest descent = direction of negative gradient

Illustration of Gradient Descent (II)



Univariate Linear Regression – GD

$$\frac{\partial}{\partial w_0} Loss(h_{\mathbf{w}}) = -2(y - h_{\mathbf{w}}(x))$$

$$\frac{\partial}{\partial w_1} Loss(h_{\mathbf{w}}) = -2(y - h_{\mathbf{w}}(x)) \times x$$

Updates for one training example (x_j, y_j):

- $w_0^t \leftarrow w_0^{t-1} + \alpha \left(y_j - h_w(x_j)\right)$
- $w_1^t \leftarrow w_1^{t-1} + \alpha \left(y_j - h_w(x_j)\right) \times x_j$

Updates for *N* training examples:

- $w_0^t \leftarrow w_0^{t-1} + \alpha \sum_{j=1}^{N} \left(y_j - h_w(x_j)\right)$
- $w_1^t \leftarrow w_1^{t-1} + \alpha \sum_{j=1}^{N} \left(y_j - h_w(x_j)\right) \times x_j$

Univariate Linear Regression – Example (III)

- **Training data:** {(1,3),(2.1,0.5),(−5,6.2)}
- **Linear regression model:** $h_{\mathbf{w}} = w_1 x + w_0$
- **Loss function:**

$$Loss(\boldsymbol{w}) = (3 - (1w_1 + w_0))^2 + (0.5 - (2.1w_1 + w_0))^2 + (6.2 - (-5w_1 + w_0))^2$$

- **The gradient:**

$$\nabla_{\mathbf{w}} Loss(\mathbf{w}) = \begin{bmatrix} \frac{\partial Loss}{\partial w_0} \\ \frac{\partial Loss}{\partial w_1} \end{bmatrix} = \begin{bmatrix} \sum_{j=1}^{3} [-2(y_j - h_w(x_j))] \\ \sum_{j=1}^{3} [-2(y_j - h_w(x_j)) \times x_j] \end{bmatrix}$$

$$= -2 \times \begin{bmatrix} (3 - w_1 - w_0) + (0.5 - 2.1w_1 - w_0) + (6.2 + 5w_1 - w_0) \\ 1(3 - w_1 - w_0) + 2.1(0.5 - 2.1w_1 - w_0) - 5(6.2 + 5w_1 - w_0) \end{bmatrix}$$

Multivariable Linear Regression – GD

$$h_{\mathbf{w}}(\mathbf{x}_j) = w_0 + w_1 x_{j,1} + \cdots + w_m x_{j,m} = w_0 + \sum_{i=1}^{m} w_i x_{j,i}$$

We set $x_{j,0} = 1$ for all the samples $j=1, \ldots, N$

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \sum_{i=0}^{m} w_i x_{j,i}$$

For one training example $(\mathbf{x}_j, y_j)$:

$$w_i^t \leftarrow w_i^{t-1} + \alpha \left(y_j - h_{\mathbf{w}}(\mathbf{x}_j) \right) \times x_{j,i}$$

For *N* training examples:

$$w_i^t \leftarrow w_i^{t-1} + \alpha \sum_{j=1}^{N} \left(y_j - h_{\mathbf{w}}(\mathbf{x}_j) \right) \times x_{j,i}$$

MONASH University
Information Technology

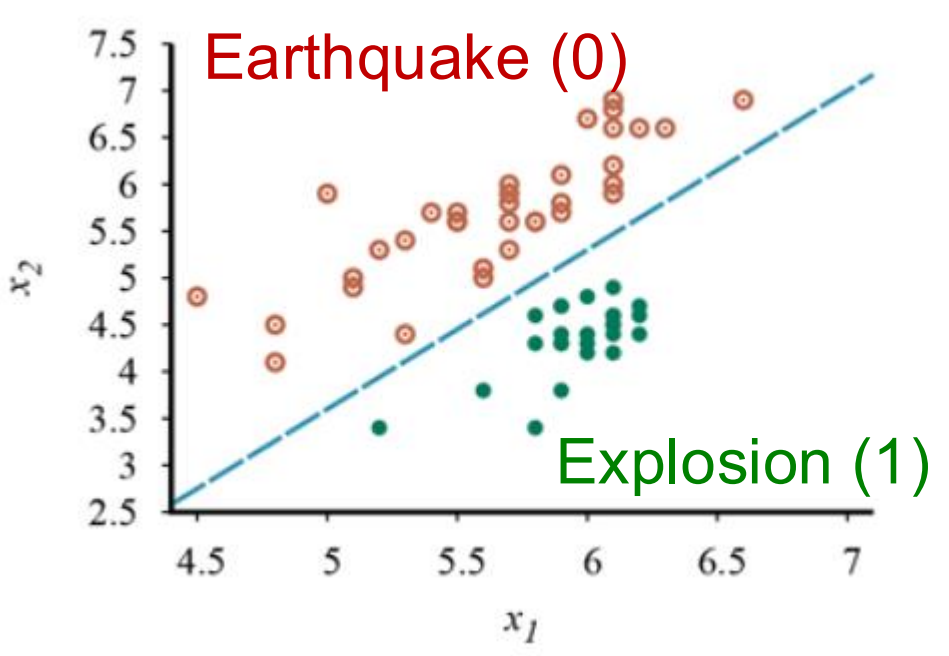


FIT3080 – Artificial Intelligence

Linear Classifiers with Thresholds

Linear Classifier (I)

- **The regressor is a *decision boundary* between two *linearly separable* classes**
  - A linear decision boundary is called a ***linear separator***
- **Example:**
  - **Linear separator**
    $-4.9 + 1.7x_1 - x_2 = 0$
  - **Classification**
    - Explosions:
      $x_2 \leq -4.9 + 1.7x_1$
    - Earthquakes:
      $x_2 > -4.9 + 1.7x_1$



Linear Classifier (II)

- Using the convention of a dummy input $x_0 = 1$, we can write the classification hypothesis as

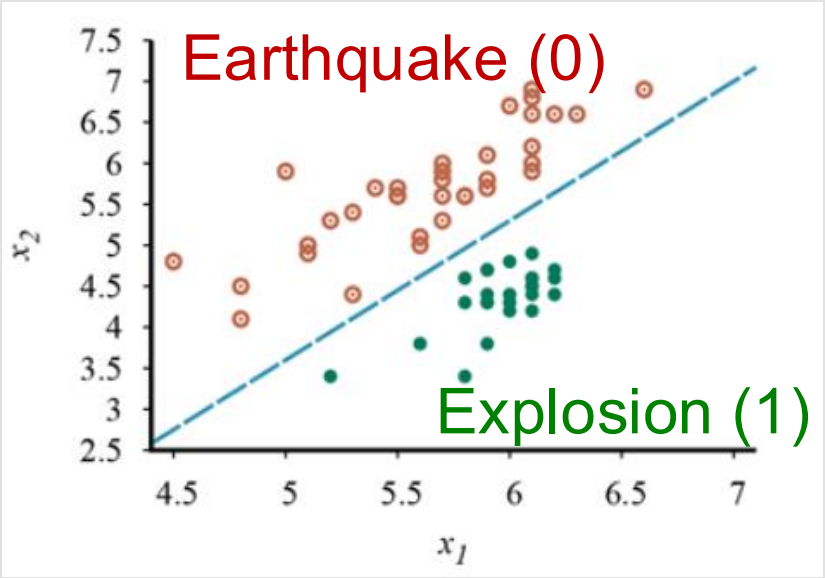
$$h_{\mathbf{w}}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where

$$\mathbf{w} = \langle -4.9, 1.7, -1 \rangle$$

to represent

$$-4.9 + 1.7x_1 - x_2 = 0$$



- Alternative representation

$$h_{\mathbf{w}}(\mathbf{x}) = Threshold(\mathbf{w} \cdot \mathbf{x})$$

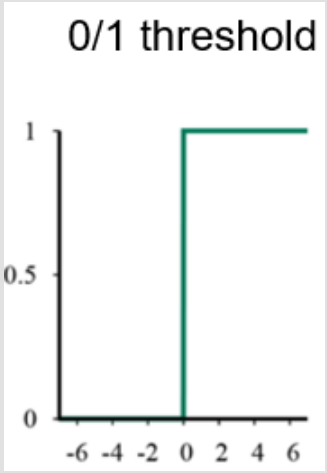
$$Threshold(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Learning the Parameters

Objective: Choose the weights $\boldsymbol{w}$ to minimize ***loss***

- We cannot perform gradient descent
- **Perceptron Learning Rule:** a weight update rule that converges on the solution, *provided the data are linearly separable*
  - For a single example $(\boldsymbol{x},y)$, where y is 0 or 1

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x}))\times x_i$$

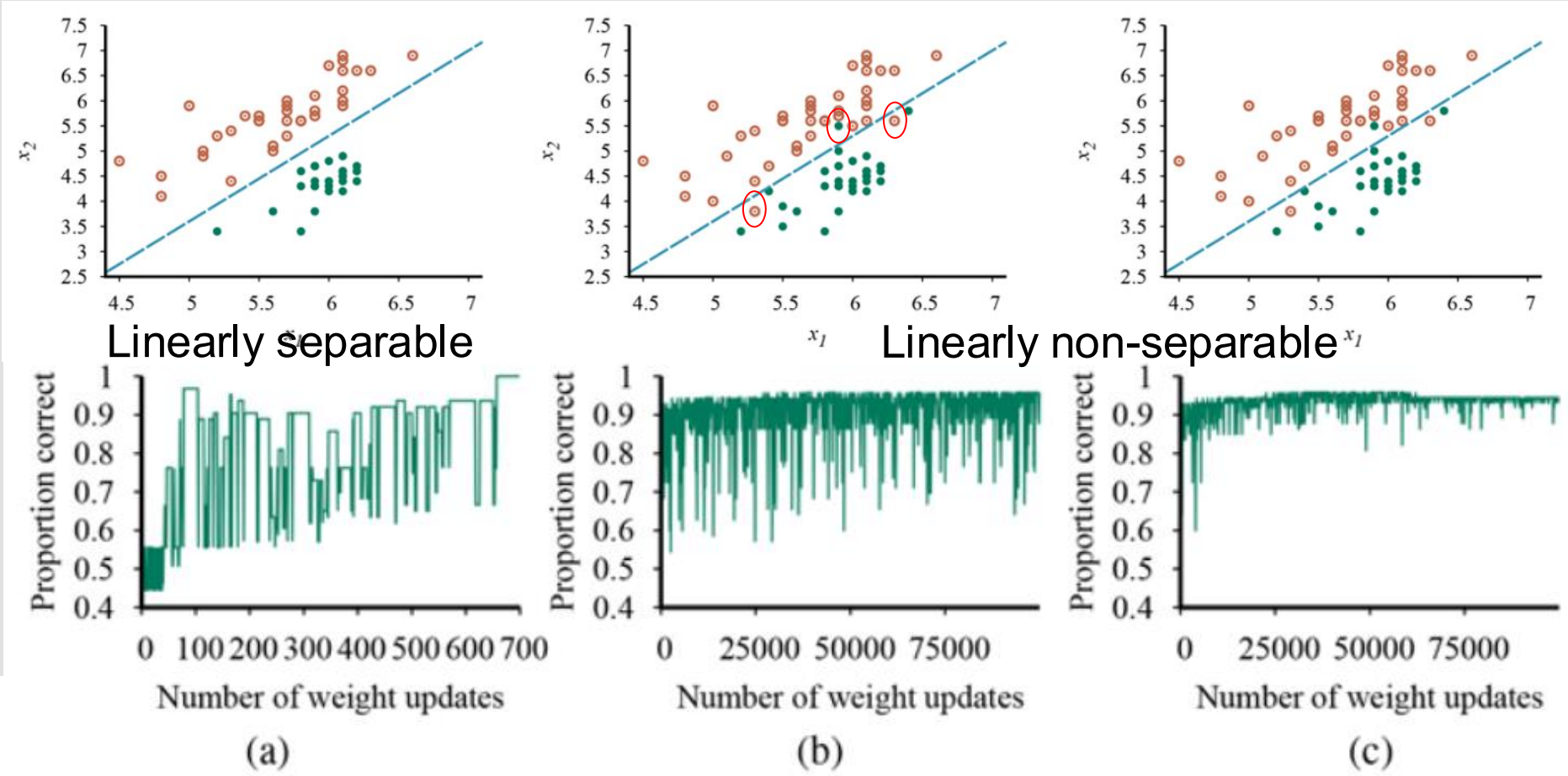


	x_i **is positive**	x_i **is negative**
$y = 1$ & $h_{\boldsymbol{w}} = 0$	w_i is increased	w_i is decreased
$y = 0$ & $h_{\boldsymbol{w}} = 1$	w_i is decreased	w_i is increased

  - It is applied one example at a time, choosing examples at random

Learning a Classifier – Training Curves

- A training curve measures the classifier performance on a fixed training set as the learning process proceeds one update at a time



Linear Classifier – Disadvantages

- The hypothesis $h_{\mathbf{w}}(\mathbf{x})$ is not differentiable, and is a discontinuous function of its inputs and its weights
  - ➔ This makes learning with the perceptron rule unpredictable
- The linear classifier always announces a completely confident prediction of 1 or 0

MONASH University
Information Technology



FIT3080 – Artificial Intelligence

Logistic Regression

Logistic Function

- A function where the linear regressor is the log of the odds of a sample belonging to the class labelled 1:

odds $\log_e\left(\frac{p}{1-p}\right) = w_0 + w_1 x_1$ ➔ $p = \frac{1}{1+ e^{-(w_0+w_1 x_1)}}$

- In general, $Logistic(z) = \frac{1}{1 + e^{-z}}$

  and for the threshold function we get

$$h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}\cdot\mathbf{x}}}$$

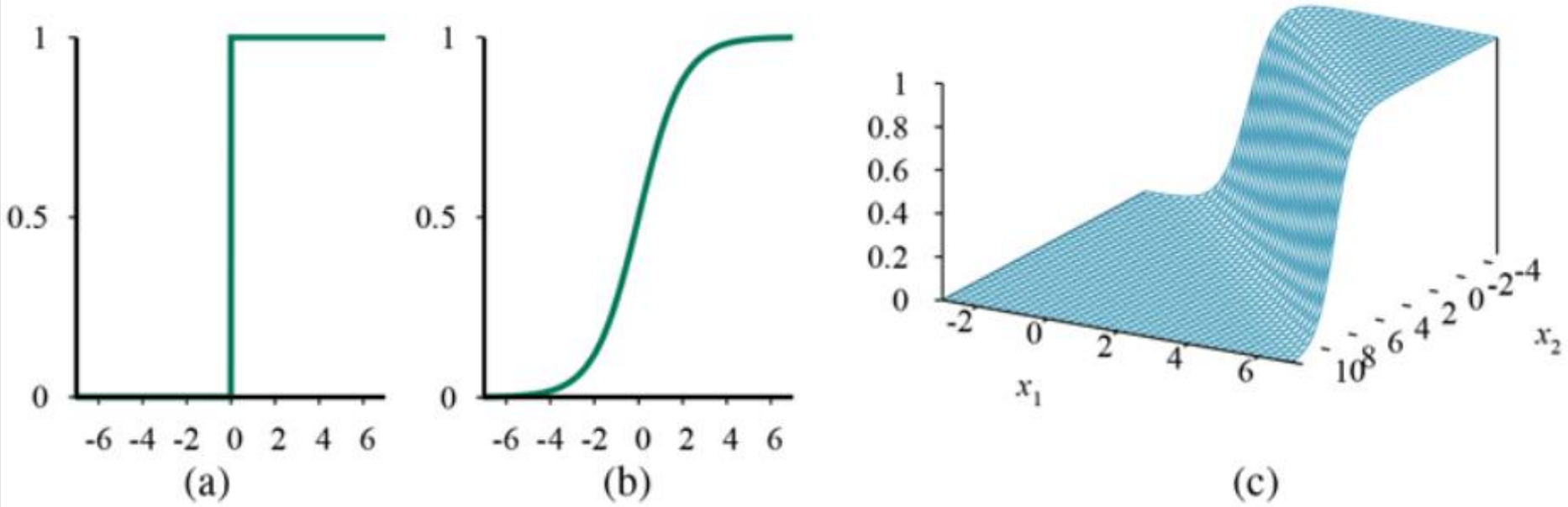
- The process of fitting the weights of this model to minimize loss on a data set is called ***logistic regression***
  - Calculated by gradient descent

Logistic and Linear Thresholds

0/1 threshold

logistic (sigmoid) function

logistic regression function for the example



Learning the Parameters (GD)

$$h_{\mathbf{w}}(\boldsymbol{x}) = Logistic(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

- **Gradient of the Loss function**

vector

$$\nabla_{\mathbf{w}} Loss(\mathbf{w}) = \left[\frac{\partial}{\partial w_i} Loss(\mathbf{w})\right] = \left[\frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x}))^2\right]$$

- **Adjusting the weights**

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} Loss(\mathbf{w})$$

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\boldsymbol{x})) \times h_{\mathbf{w}}(\boldsymbol{x}) \times (1 - h_{\mathbf{w}}(\boldsymbol{x})) \times x_i$$

Learning rate

$$\Delta = \alpha \times Error \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(\mathbf{x}))$$

Learning Weights – Earthquake Example

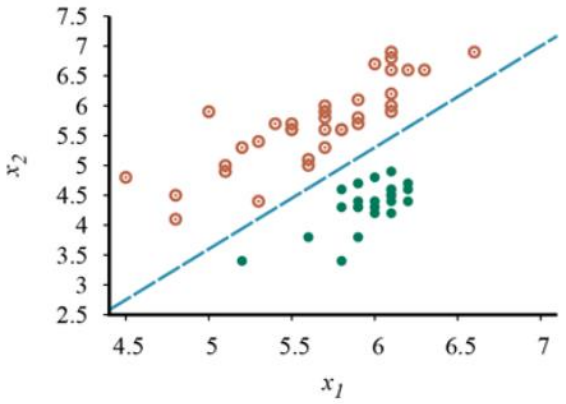
$$h_{\mathbf{w}}(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}$$

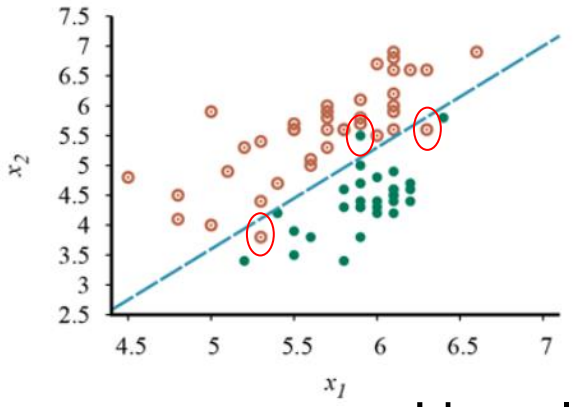
$$w_i \leftarrow w_i + \underbrace{\alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times h_{\mathbf{w}}(\mathbf{x}) \times (1 - h_{\mathbf{w}}(\mathbf{x}))}_{\Delta} \times x_i$$

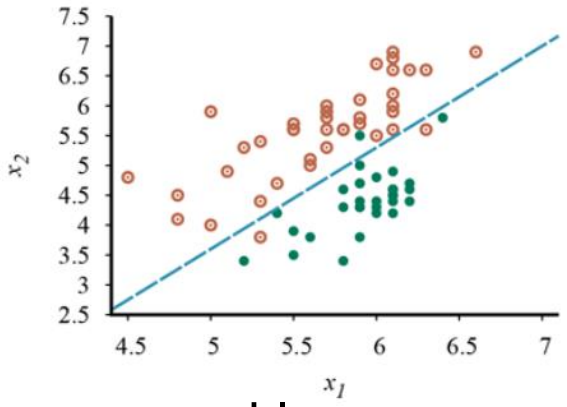
$\alpha = 0.2$

t	w_0	w_1	w_2	x_0	x_1	x_2	$\mathbf{w} \cdot \mathbf{x}$	$h_{\mathbf{w}}(\mathbf{x})$	y	Δ
0	0	0	0	1	5.2	3.4	0	0.5	1	0.025
1	0.025	0.13	0.085	1	5	6	1.185	0.765846	0	-0.02747
2	-0.00247	-0.00734	-0.0798	1	6	6.6	-0.57319	0.360502	0	-0.01662
3	-0.01909	-0.10707	-0.18951	1	6	4.9	-1.59009	0.169371	1	0.023371

Learning the Classifier – Training Curves

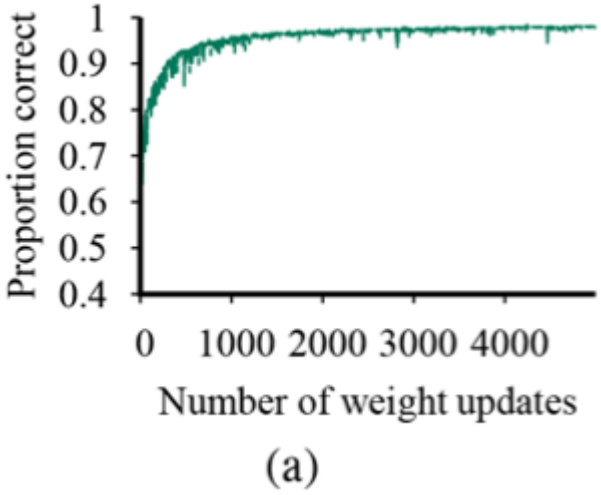


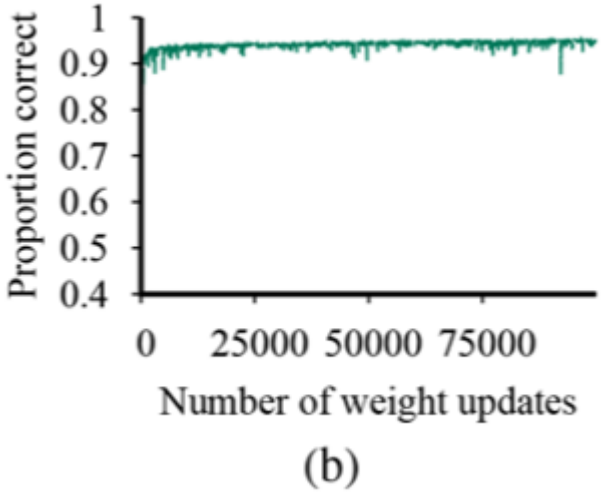


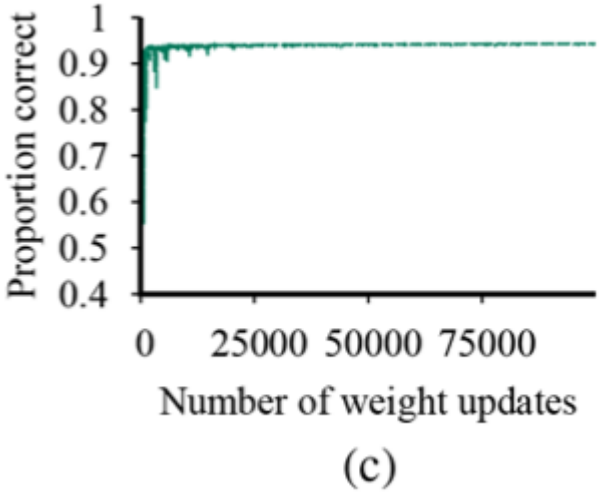


Linearly separable

Linearly non-separable







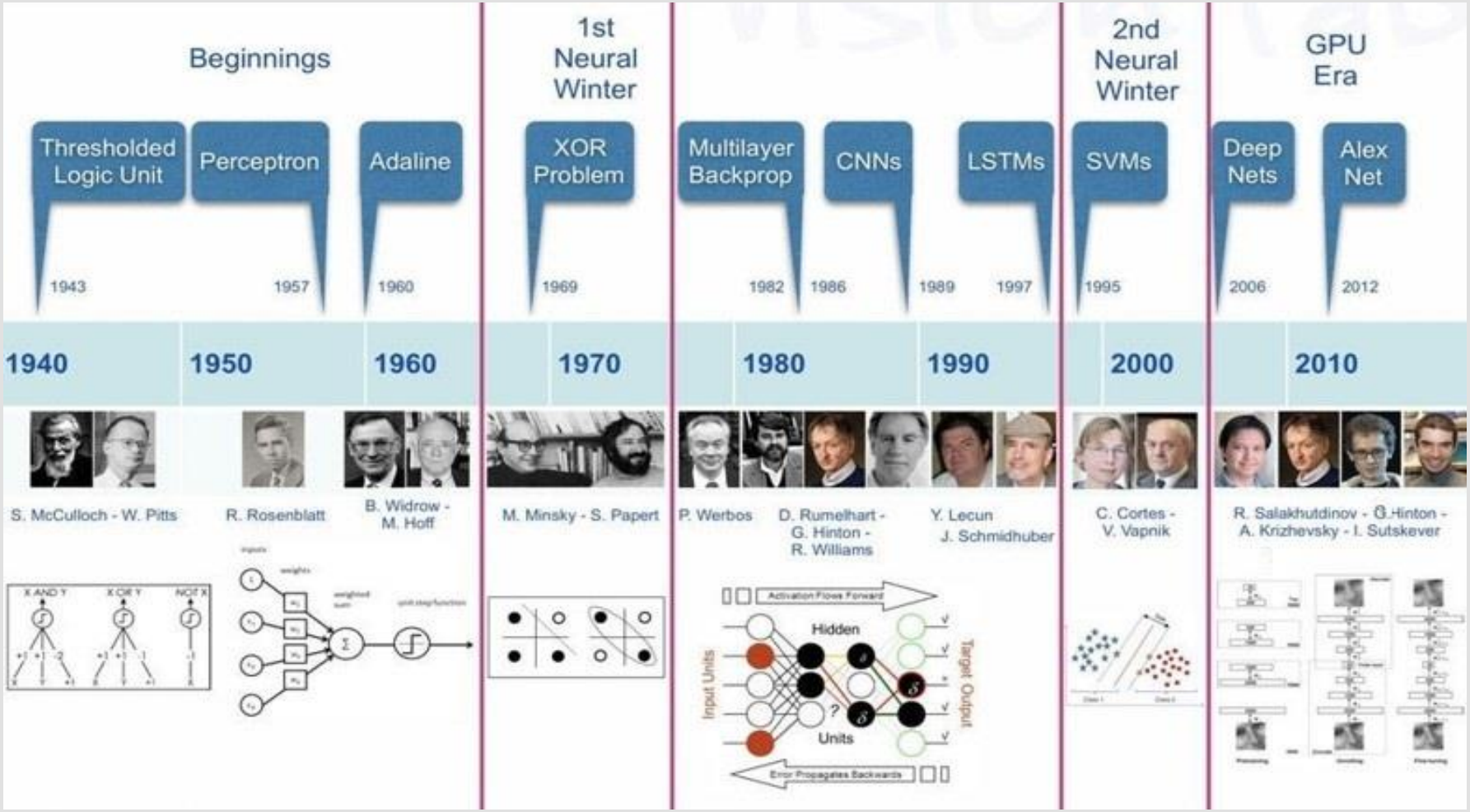
MONASH University
Information Technology



FIT3080 – Artificial Intelligence

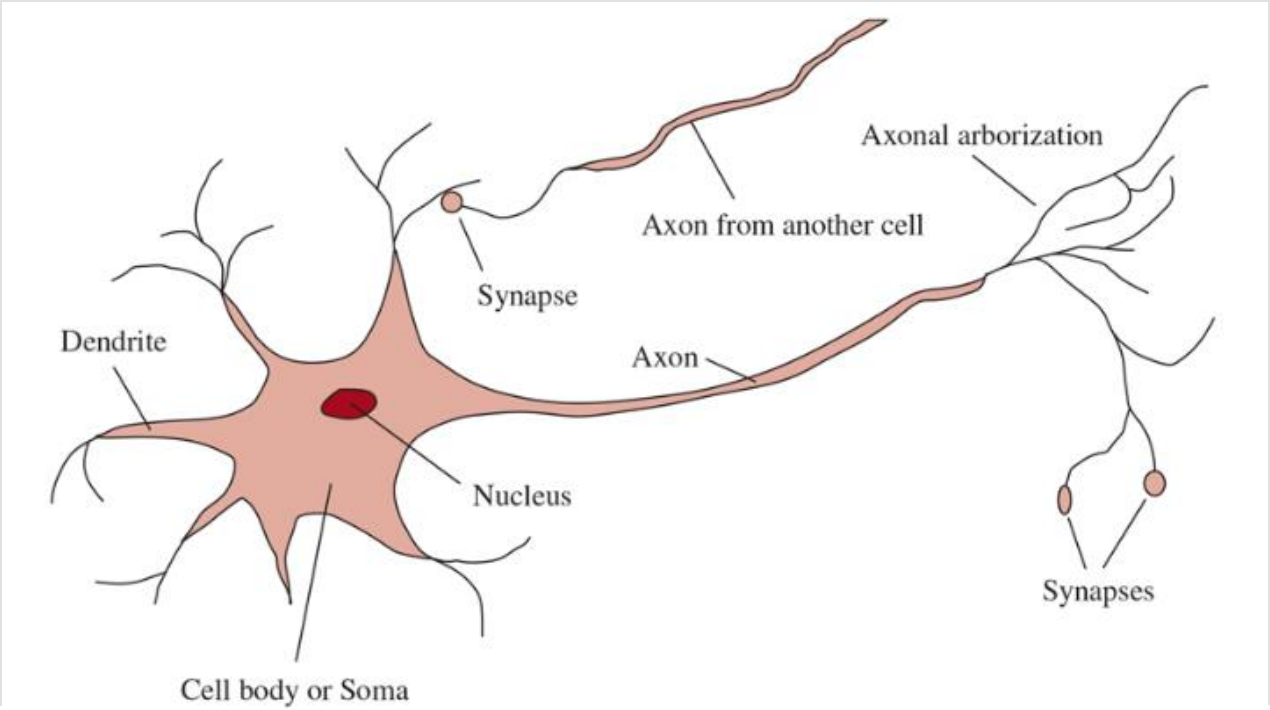
Artificial Neural Networks

History of Artificial Neural Network



Brains

- 10^{11} neurons of > 20 types
- 10^{14} synapses
- 1ms–10ms cycle time
- Signals are noisy “spike trains” of electrical potential

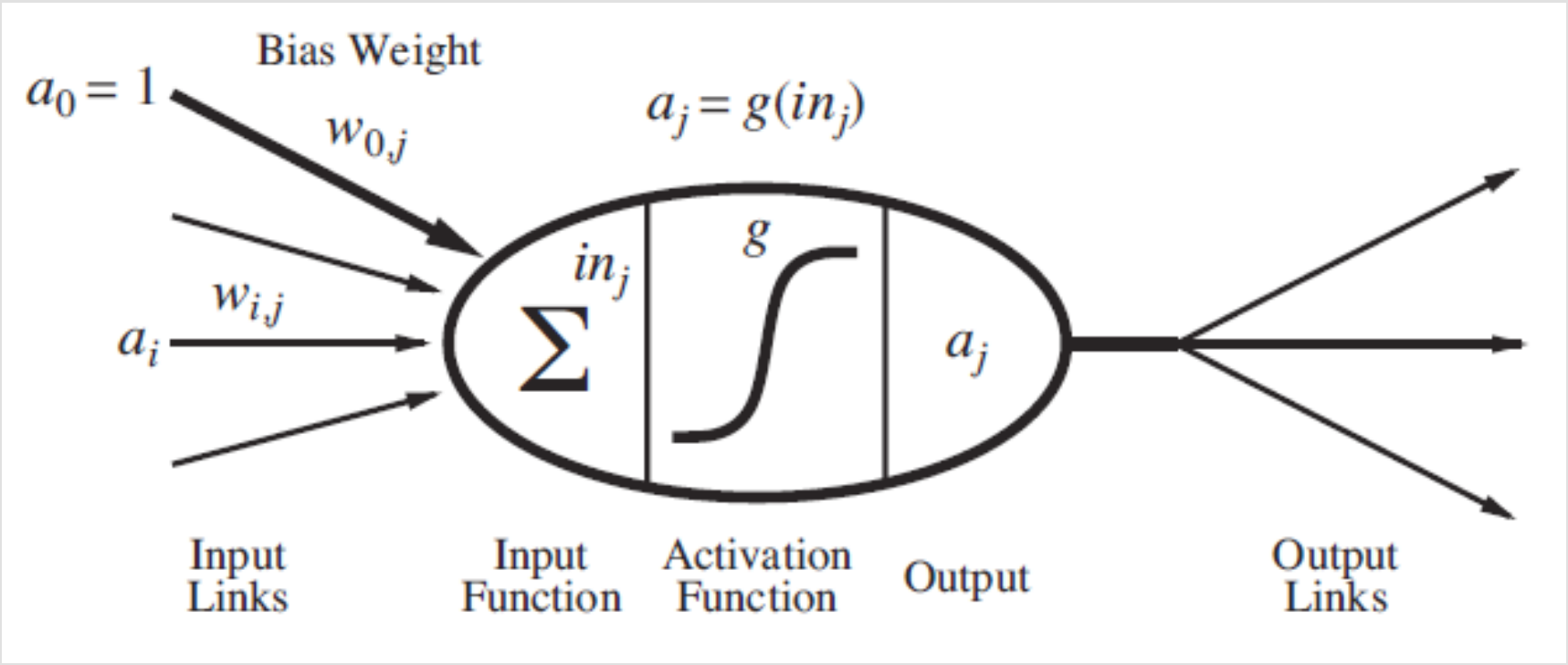


McCulloch-Pitts (1943) – Simplified Neuron

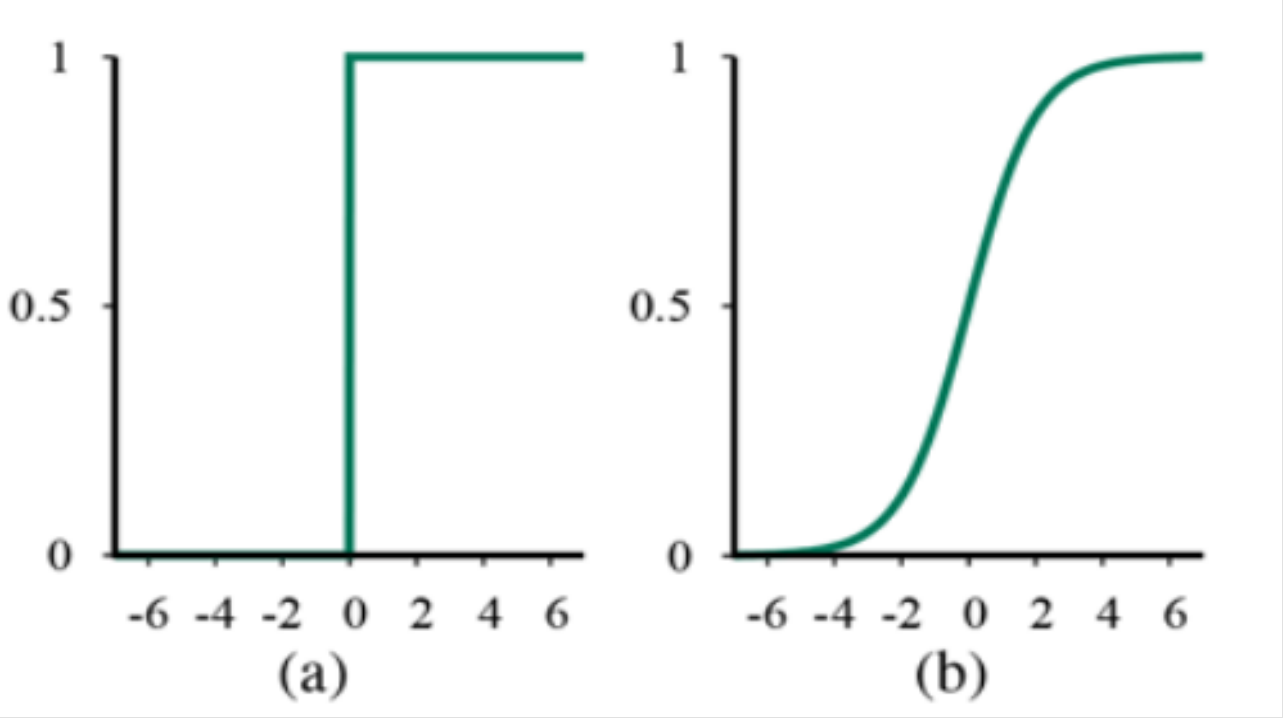
- Output is a function of the inputs:

$$a_j = g(in_j) = g(\mathbf{w}_j \cdot \mathbf{a}) = g\left(\sum_{i=0}^{n} w_{i,j}\, a_i\right)$$

activation function



Activation Functions



$$g(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else} \end{cases}$$

Perceptron

$$g(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid perceptron

Network Structure (I)

- **Feed-forward network**
  - Has connections only in one direction
  - Represents a function of its current input
    → Has no internal state
  - May have several outputs
  - Types
    - Single-layer perceptrons
    - Multi-layer networks
      - Arranged in **layers** – each unit receives input only from units in the immediately preceding layer
      - Have one or more **layers** of **hidden units** that are ***not*** connected to outputs of the network

Network Structure (II)

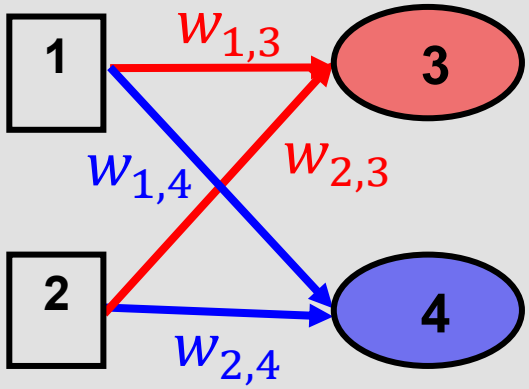
- **Recurrent network**
  - Feeds its outputs back into its own inputs
    - the network's response to an input depends on its initial state, which may depend on previous inputs
      → Can support short-term memory
  - Has an internal state

Single-layer Feed-forward NN (Perceptrons)

- **A network with all the inputs connected directly to the outputs**
  - When there are *m* outputs
    - Each output unit corresponds to a separate function
    - Each weight affects only one output
      → there are *m* training processes
- **Example:**

$$a_3 = w_{1,3}a_1 + w_{2,3}a_2$$

$$a_4 = w_{1,4}a_1 + w_{2,4}a_2$$



Example: Spam Filter (I)

- **Input: email**
- **Output: SPAM/HAM**
- **Setup:**
  - Get a large collection of sample emails, each labeled "SPAM" or "HAM"
    - The samples have been labeled
  - **Goal**: learn to predict labels for new, future emails
- **Features: The attributes used to make a prediction**
  - words in the email's text



Dear Sir.
First, I must solicit your confidence in this transaction, this is by virture of its nature as being utterly confidencial and top secret. …



TO BE REMOVED FROM FUTURE MAILINGS, SIMPLY REPLY TO THIS MESSAGE AND PUT "REMOVE" IN THE SUBJECT.
99 MILLION EMAIL ADDRESSES FOR ONLY $99



Ok, Iknow this is blatantly OT but I'm beginning to go insane. Had an old Dell Dimension XPS sitting in the corner and decided to put it to use, I know it was working pre being stuck in the corner, but when I plugged it in, hit the power nothing happened.

Example: Spam Filter (II)

- **Four features**
  - BIAS (always has value 1)
  - free (# occurrences of “free”)
  - money (# occurrences of “money”)
  - the (# occurrences of “the”)

$$w \cdot f(x) = \sum_i w_i f(x_i)$$

x	$f(x)$	w	
“free money”	BIAS : 1 free : 1 money : 1 the : 0 ...	BIAS : -3 free : 4 money : 2 the : 0 ...	-3*1 + 4*1 + 2*1 + 0*0 … =3

Prediction: Binary Decision Rule

- **In the space of feature vectors**
  - Any weight vector is a hyperplane
    - One side will be class 1
    - Other side will be class 0
- **Example: Spam filter**

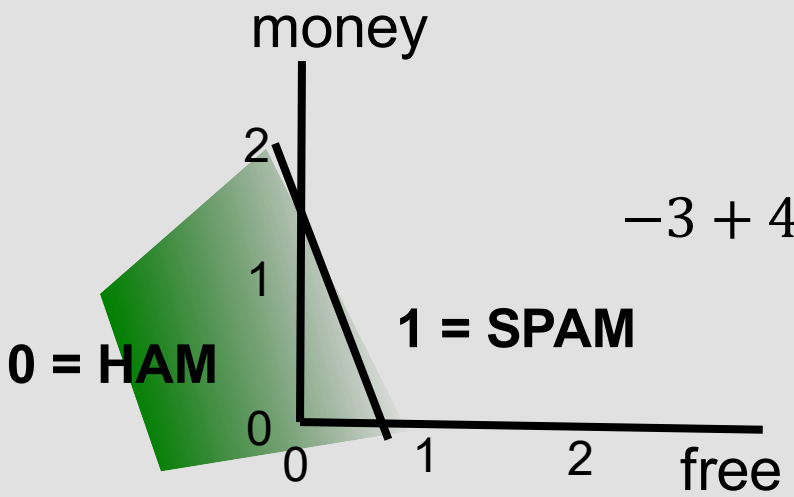
$$g(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \boldsymbol{f}(\boldsymbol{x}) \geq 0 \\ 0 & \text{else} \end{cases}$$

$$\boldsymbol{w} \cdot f(\boldsymbol{x}) = 0$$

$$-3 + 4free + 2money = 0$$

$\boldsymbol{w}$

BIAS :	-3
free :	4
money :	2
the :	0
...	



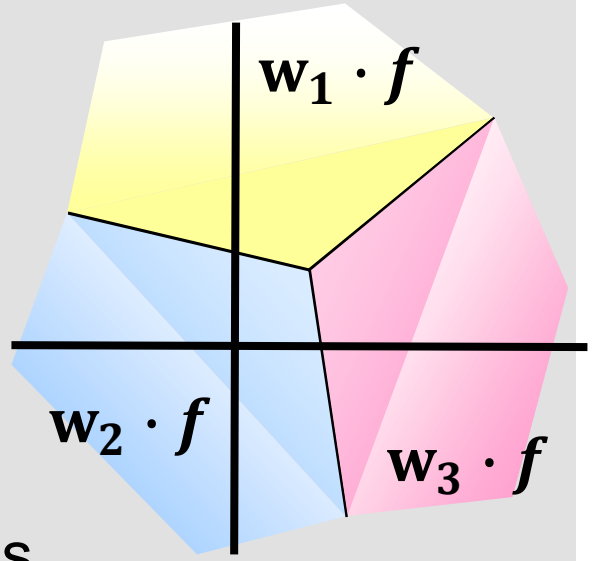
Prediction: Multiclass Decision Rule

- **For more than two classes**
  - Have a weight vector for each class
  - Calculate the activation for each class

$$\text{activation}_w(x, c) = \sum_i w_{c,i} \cdot f_i(x)$$

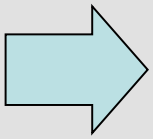
  - The class with the highest activation wins

$$c = \arg\max_c \ (\text{activation}_w(x, c))$$



Example: Multiclass Decision Rule

“win the vote”



```
BIAS   :  1
win    :  1
game   :  0
vote   :  1
the    :  1
...
```

w_{SPORTS}

```
BIAS   : -2
win    :  4
game   :  4
vote   :  0
the    :  0
...
```

$w_{POLITICS}$

```
BIAS   :  1
win    :  2
game   :  0
vote   :  4
the    :  0
...
```

w_{TECH}

```
BIAS   :  2
win    :  0
game   :  2
vote   :  0
the    :  0
...
```

Learning Perceptrons

- **Each weight affects only one output**
  → there are *m* training processes
- **Depending on the activation function, the training process will be either**
  - **Perceptron:** perceptron learning rule

$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$

α is the learning rate

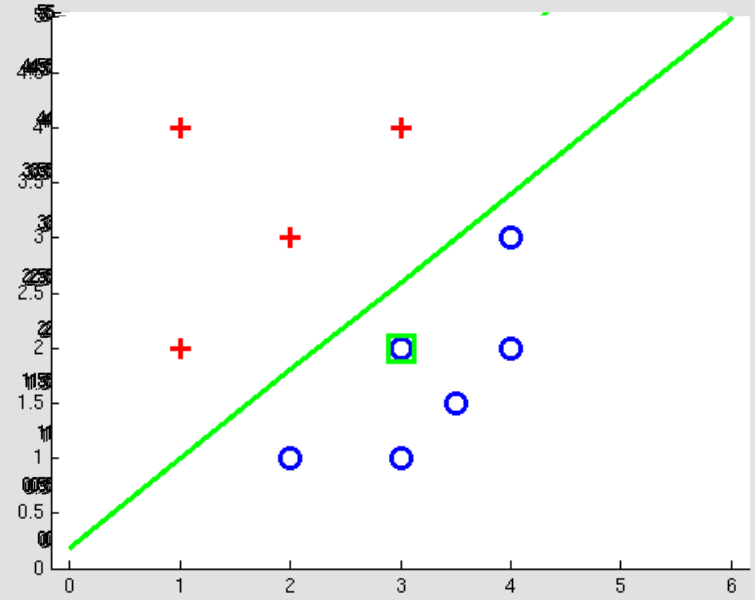
	x_i **is positive**	x_i **is negative**
$y = 1$ & $h_{\mathbf{w}} = 0$	w_i is increased	w_i is decreased
$y = 0$ & $h_{\mathbf{w}} = 1$	w_i is decreased	w_i is increased

  - **Sigmoid perceptron:** gradient descent rule for **logistic regression**

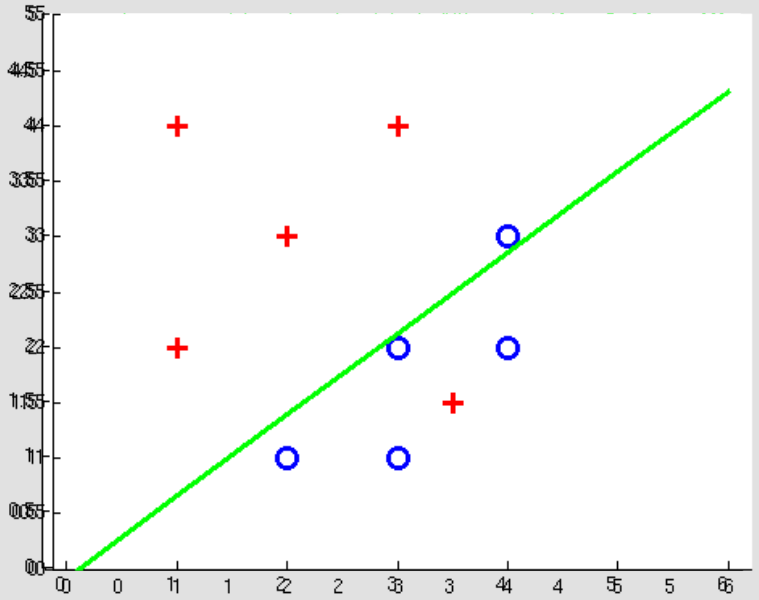
$$w_i \leftarrow w_i + \underbrace{\alpha(y - h_{\mathbf{w}}(\mathbf{x}))}_{Error} \times \underbrace{h_{\mathbf{w}}(\mathbf{x}) \times (1 - h_{\mathbf{w}}(\mathbf{x}))}_{g'(\mathbf{w} \cdot \mathbf{x})} \times x_i$$

Perceptrons – Separability

- **Separable Case**

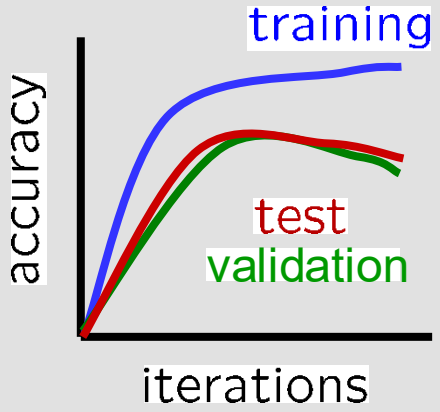


- **Non-Separable Case**

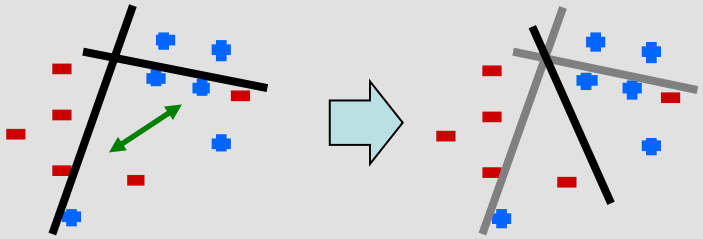


Issues with Perceptrons

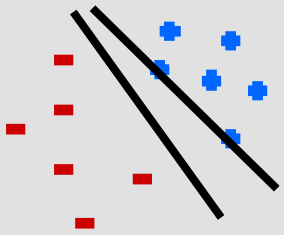
- **Overtraining**
  - Test / validation accuracy usually rises, then falls



- **Non-separability**
  - If the data aren't separable, weights might thrash around



- **Mediocre generalization**
  - Finds a "barely" separating solution

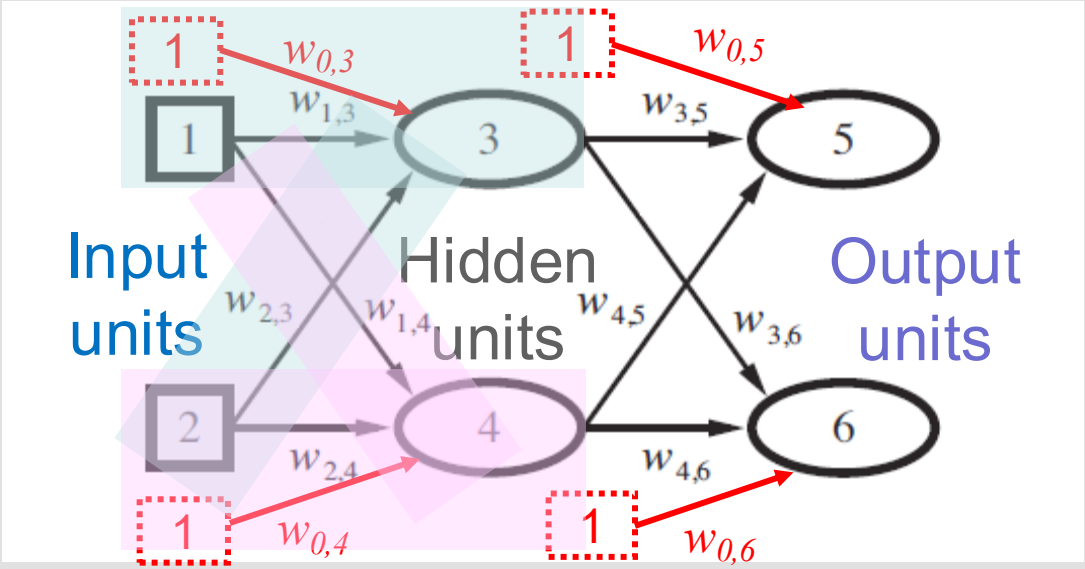


Multi-layer Feed-forward NN

- **A network with one or more hidden layers**
  - Given an input vector $\mathbf{x} = (x_1, x_2)$
    - The activations of the input units are set to $(a_1, a_2) = (x_1, x_2)$
    - The output of unit 5 is

$$a_5 = g(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4)$$

$$a_3 = g(w_{0,3} + w_{1,3}a_1 + w_{2,3}a_2) \qquad a_4 = g(w_{0,4} + w_{1,4}a_1 + w_{2,4}a_2)$$

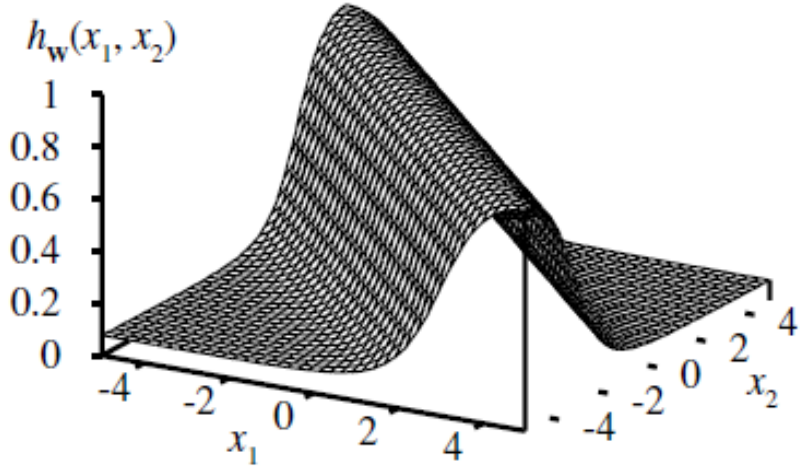


Dummy inputs = 1

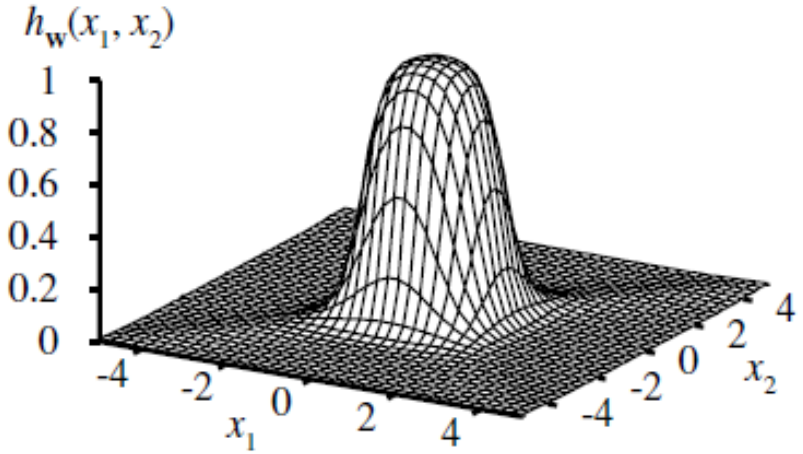
Expressiveness of Multi-layer FF NNs

- **Each unit in a sigmoid network represents a soft threshold**
- **Can create arbitrarily complex functions**

Two opposite-facing soft threshold functions



Four "opposite facing" soft threshold functions at right angles



Learning Multi-layer Feed-forward NNs

- **Problems:**
  - Interactions among learning problems when the network has multiple outputs
    - For an *additive* loss function, we add the gradients of the individual losses

$$\frac{\partial}{\partial w} Loss(\boldsymbol{w}) = \frac{\partial}{\partial w} \sum_{k=1}^{m} (y_k - a_k)^2 = \sum_{k=1}^{m} \frac{\partial}{\partial w} (y_k - a_k)^2$$

nodes in the output layer

➔ we can decompose an *m*-output learning problem into *m* learning problems: when updating each weight, add up the gradient contributions from each output

  - How do we calculate the error in the hidden layers?
    - **Back-propagation**

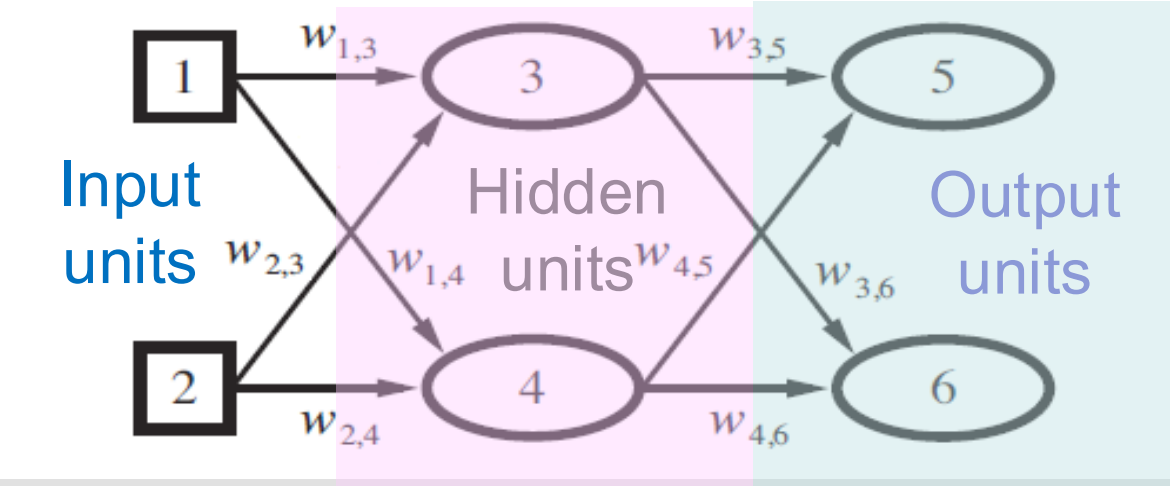
Back-propagation

- Weight adjustment hidden unit $j \rightarrow$ output unit k

$$\Delta_k \leftarrow (y_k - a_k) g'(in_k) \qquad w_{j,k} \leftarrow w_{j,k} + \alpha \times a_j \times \Delta_k$$

- Weight adjustment input/hidden unit $i \rightarrow$ hidden unit j

$$\Delta_j \leftarrow g'(in_j) \boxed{\sum_k w_{j,k} \Delta_k} \qquad w_{i,j} \leftarrow w_{i,j} + \alpha \times a_i \times \Delta_j$$



Back-propagation Algorithm

1. Compute the Δ values for the output units, using the observed error
2. Starting with the output layer, for each layer in the network **do** until the earliest hidden layer is reached
   a. Propagate the Δ values back to the previous layer
   b. Update the weights between the two layers

MONASH University
Information Technology

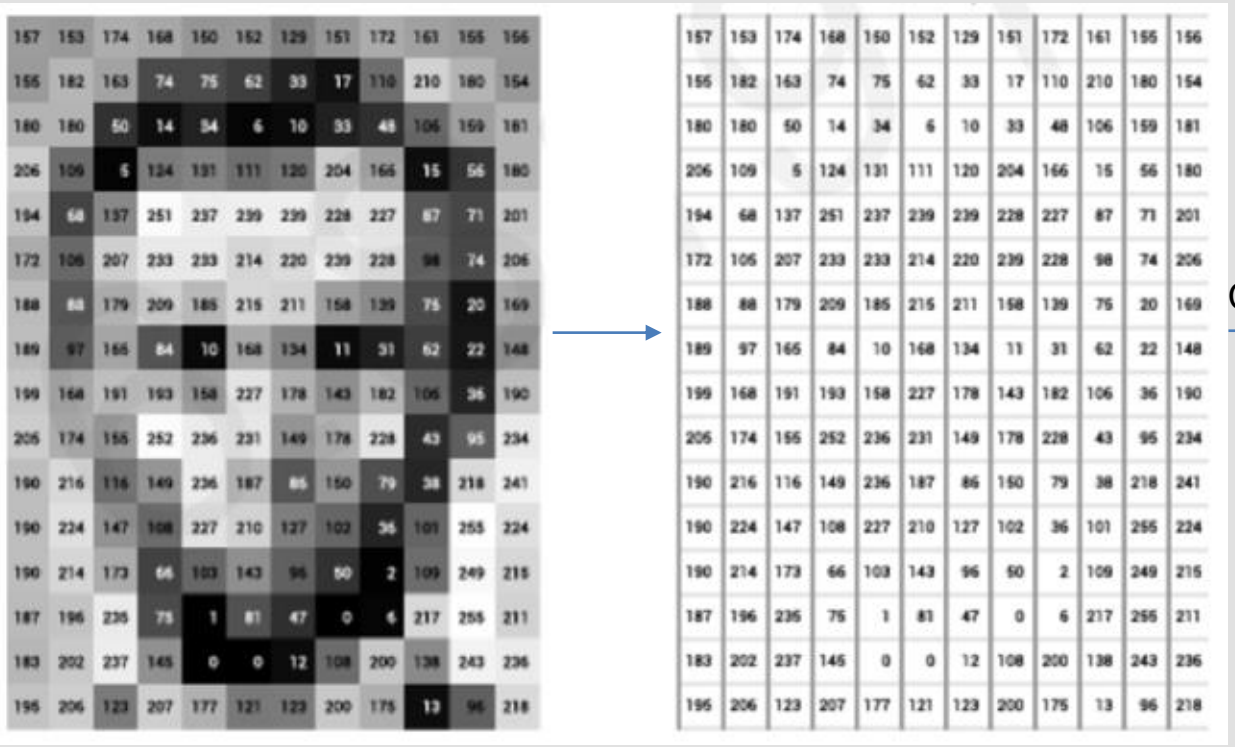


FIT3080 – Artificial Intelligence

Convolutional Neural Networks

Convolutional Neural Network (CNN)

- **What computers "see"**
  - Images are numbers



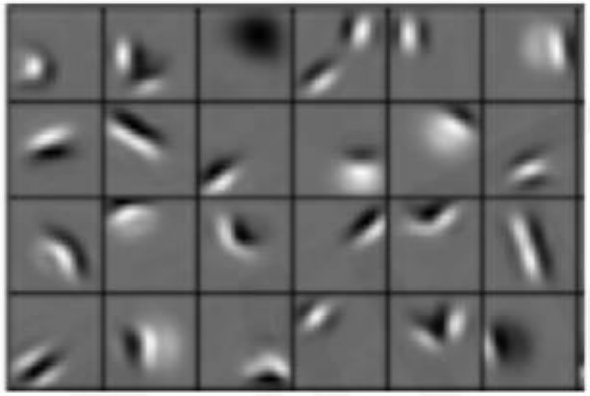
Input Image

Pixel Representation

Convolutional Neural Network (CNN)

- **Can we learn a hierarchy of features directly from the data?**

Low level features



Edges, dark spots

Mid level features



Eyes, ears, nose

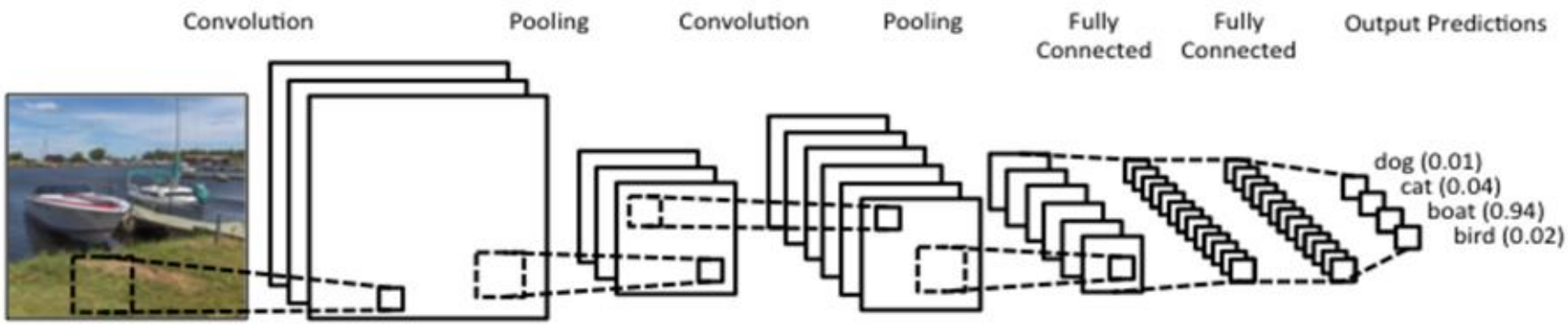
High level features



Facial structure

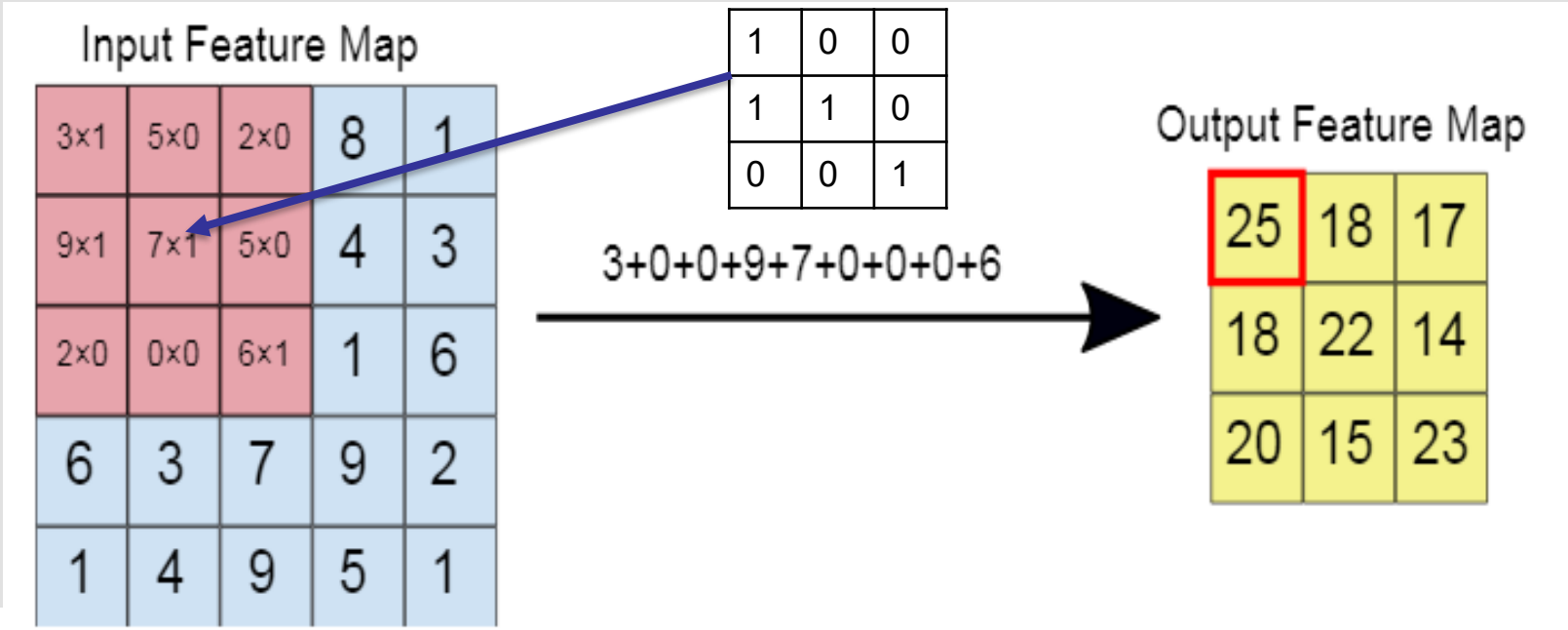
Convolutional Neural Network (CNN)

- **Architecture of a traditional CNN**
  - generally composed of the following layers:
    - Convolutions
    - ReLU
    - Pooling
    - Fully Connected



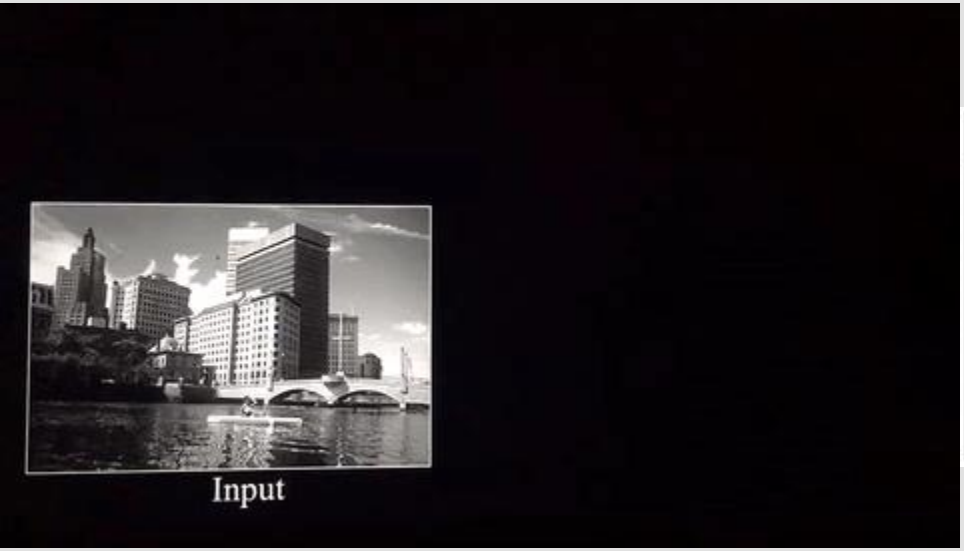
Convolutional Neural Network (CNN)

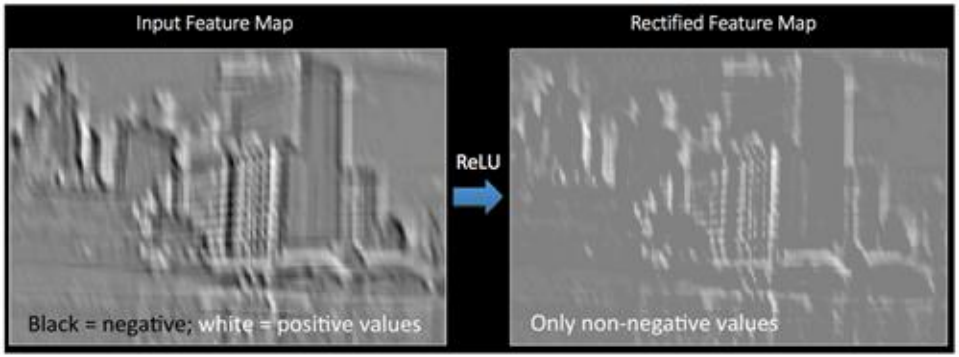
- **Convolution layer (CONV)**
  - uses filters that perform convolution operations
  - Its hyperparameters include:
    - > the filter size, F and stride S
  - Resulting output, O: feature map or activation map



Convolutional Neural Network (CNN)

- **Introducing non-linearity**
  - Apply after every convolution operation
  - Rectified Linear Unit (ReLU):
    > Pixel by pixel operation that replaces all negative values by zero
  - Example on real image:

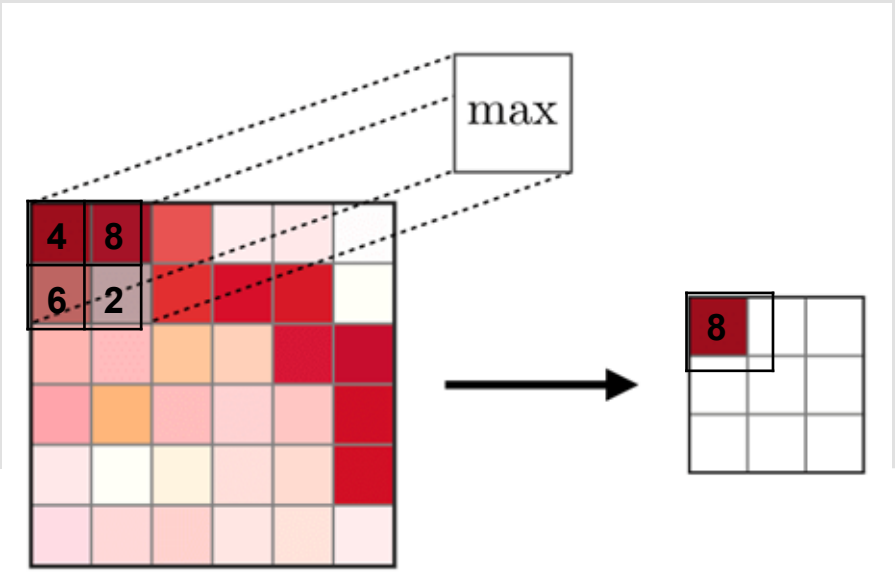


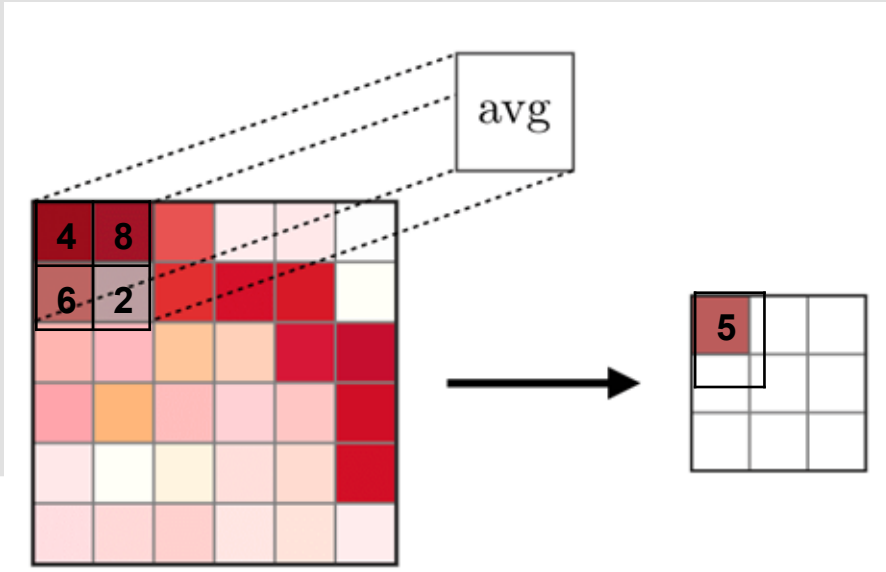


Feature map from convolution operation

Convolutional Neural Network (CNN)

- **Pooling layer (POOL)**
  - Is a down-sampling operation,
    - > typically applied after a convolution layer,
    - > To reduce the dimensionality and does some spatial invariance.
  - Types of pooling
    - > Max pooling: selects the maximum value
    - > Average pooling: averages the values

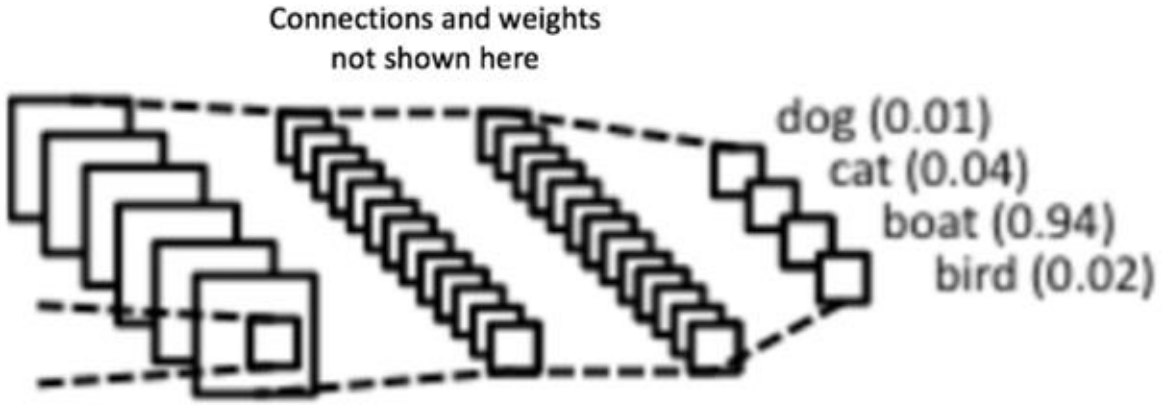




Convolutional Neural Network (CNN)

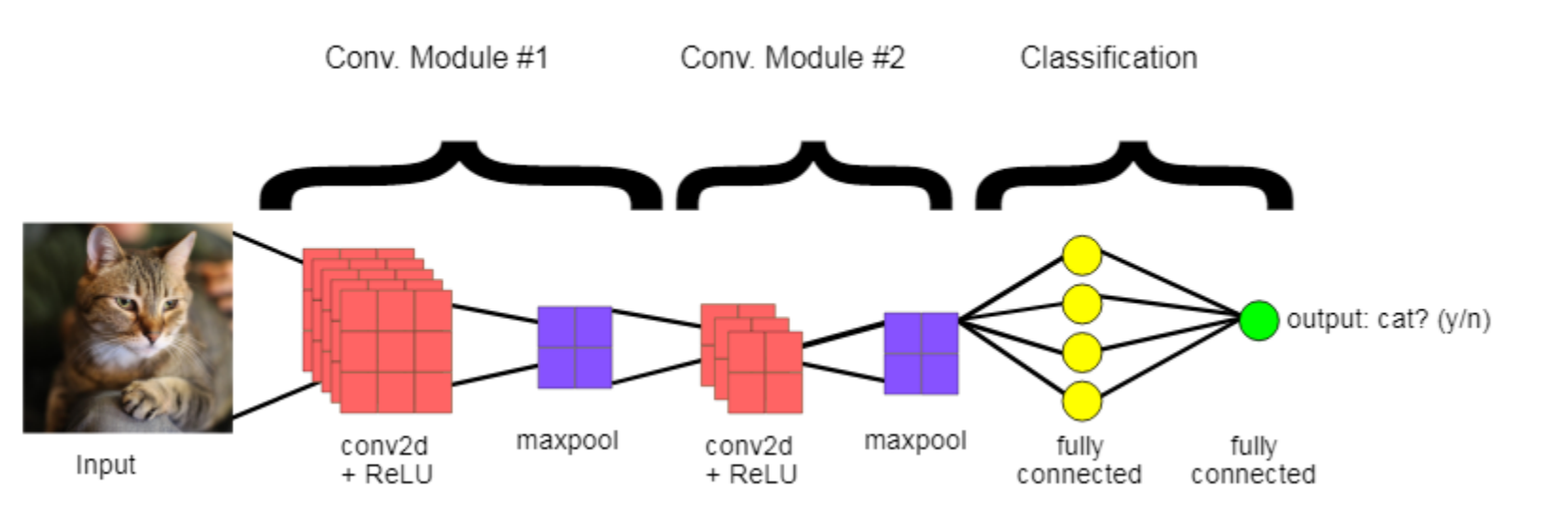
- **Fully connected layer**
  - One or more fully connected layers at the end of CNN
  - To perform classification based on the features extracted by the convolutions
  - "Fully Connected" implies that
    - every neuron in the previous layer is connected to every neuron on the next layer
  - Typically, the final fully connected layer contains a softmax activation function,
    - which outputs a probability value from 0 to 1

Example:



Convolutional Neural Network (CNN)

- **Overview of CNN**





Reading

- **Supervised learning –** Russell, S. and Norvig, P. (2020), *Artificial Intelligence – A Modern Approach* (4th ed), Prentice Hall
  - Regression, Logistic regression – Chapter 19.6.1-19.6.4
  - Artificial Neural Networks – Chapter 22.1-22.3

Next Lecture Topic

- **Lecture Topic LN10**
  - Machine Learning Part 3 –
  - Naïve Bayes, Smoothing, k nearest neighbour, ensemble learning, unsupervised learning